K-means (Метод k-средних)

Метод k-средних является одним из наиболее распространённых алгоритмов кластеризации, применяемых для разбиения данных на заранее заданное количество групп. Он основан на минимизации расстояния между объектами и центрами кластеров.

Суть метода

Основная идея алгоритма заключается в том, что все объекты разбиваются на k кластеров таким образом, чтобы каждая точка принадлежала тому кластеру, центр которого находится к ней ближе всего.

Центры кластеров (центроиды) не задаются заранее, а вычисляются в процессе работы алгоритма как среднее значение точек, входящих в кластер.

Таким образом, происходит итеративное уточнение положения центров и состава кластеров.

Принцип работы

Алгоритм выполняется следующим образом:

1. Задаётся количество кластеров k
2. Случайным образом выбираются начальные центры кластеров
3. Каждая точка относится к ближайшему центру
4. Пересчитываются центры кластеров как среднее значение точек
5. Процесс повторяется до сходимости

Сходимость достигается тогда, когда центры перестают существенно изменяться.

Математическая формализация

Алгоритм минимизирует следующую функцию:

$$J = \sum_{i=1}^{k} \sum_{x \in C_i} ||x - \mu_i||^2$$

где:

- Ci — i-й кластер
- μi — центр кластера
- x — объект

Таким образом, минимизируется сумма квадратов расстояний от точек до центров кластеров.

Основные параметры

- n_clusters — количество кластеров
- init — способ инициализации центров (например, k-means++)
- max_iter — максимальное число итераций
- random_state — фиксирование случайности

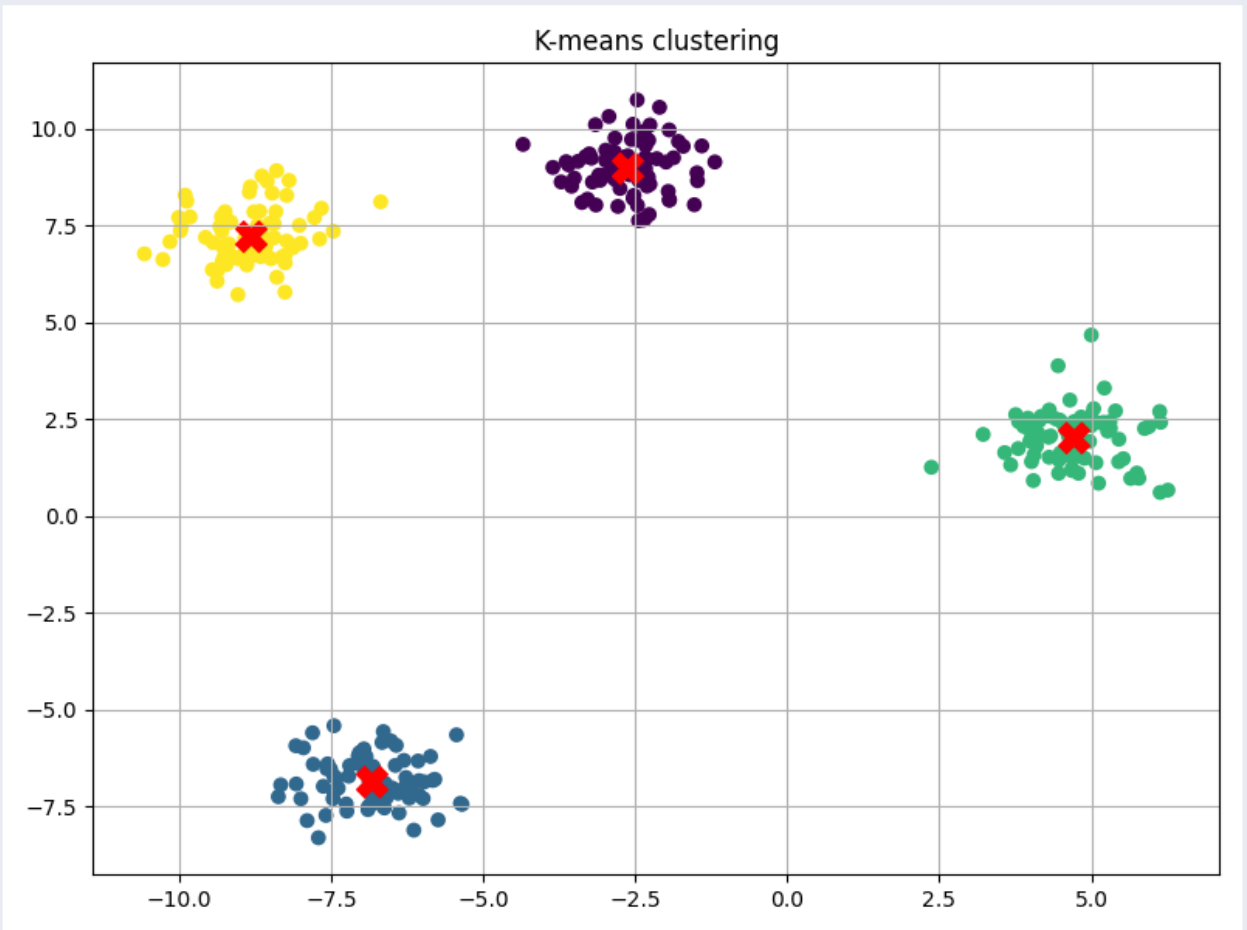
Особенности метода

- требует задания числа кластеров
- хорошо работает с компактными и сферическими кластерами
- чувствителен к начальному положению центров

Реализация на Python

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs

# Генерация данных
X, _ = make_blobs(
    n_samples=300,
    centers=4,
    cluster_std=0.7,
    random_state=42
)

# Создание модели
model = KMeans(n_clusters=4, random_state=42)

# Обучение
labels = model.fit_predict(X)

# Визуализация
plt.figure(figsize=(8, 6))
plt.scatter(X[:, 0], X[:, 1], c=labels, cmap='viridis')

# Отображение центров
centers = model.cluster_centers_
plt.scatter(centers[:, 0], centers[:, 1], c='red', s=200, marker='X')

plt.title("K-means clustering")
```

```
plt.grid(True)
plt.show()
```



Пояснение к примеру

В данном примере создаётся набор данных с четырьмя кластерами. Алгоритм K-means получает число кластеров в качестве входного параметра и далее самостоятельно определяет границы между ними.

Центры кластеров отображаются на графике и соответствуют средним значениям точек внутри каждого кластера.

Выбор числа кластеров

Одной из ключевых задач является определение оптимального значения k. Для этого часто используется метод "локтя".

Суть метода заключается в анализе зависимости суммы квадратов расстояний (inertia) от числа кластеров.

Реализация метода локтя

```
import numpy as np
from sklearn.cluster import KMeans
```

```
from sklearn.datasets import make_blobs
import matplotlib.pyplot as plt

X, _ = make_blobs(
    n_samples=400,
    centers=6,
    cluster_std=0.8,
    random_state=42
)

inertia = []

for k in range(1, 10):
    model = KMeans(n_clusters=k, random_state=42)
    model.fit(X)
    inertia.append(model.inertia_)

plt.figure(figsize=(8, 6))
plt.plot(range(1, 10), inertia, marker='o')
plt.title("Метод локтя")
plt.xlabel("Количество кластеров")
plt.ylabel("Inertia")
plt.grid(True)
plt.show()
```

Достоинства метода

1. Простота реализации
2. Высокая скорость работы
3. Хорошо масштабируется
4. Понятная интерпретация результатов

Недостатки метода

1. Требует задания числа кластеров
2. Чувствителен к выбросам
3. Плохо работает с кластерами сложной формы
4. Зависит от начальной инициализации

Задачи для проверки понимания

Задача 1. Влияние числа кластеров

Вопрос: Что произойдёт при слишком большом значении k?

Ответ:
При слишком большом k данные будут разбиты на слишком мелкие кластеры, что может привести к переобучению и потере обобщающей способности модели.

Задача 2. Начальная инициализация

Вопрос: Почему важно правильно выбирать начальные центры?

Ответ:
Поскольку алгоритм сходится к локальному минимуму, неудачный выбор начальных центров может привести к неоптимальному разбиению данных.

Задача 3. Ограничения метода

Вопрос: Почему K-means плохо работает с кластерами сложной формы?

Ответ:
Потому что алгоритм предполагает, что кластеры имеют примерно сферическую форму и равномерную плотность. При нарушении этих условий результат может быть некорректным.

Задача 4. Практический подбор k

Условие:
Дан набор данных. Требуется определить оптимальное количество кластеров.

Решение:

```
import numpy as np
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs
from sklearn.metrics import silhouette_score

X, _ = make_blobs(
    n_samples=500,
    centers=5,
    cluster_std=0.9,
    random_state=42
)

best_k = 2
best_score = -1

for k in range(2, 10):
    model = KMeans(n_clusters=k, random_state=42)
```

```
    labels = model.fit_predict(X)

    score = silhouette_score(X, labels)

    if score > best_score:
        best_score = score
        best_k = k

print(f"Оптимальное количество кластеров: {best_k}")
print(f"Silhouette Score: {best_score:.3f}")
```

Пояснение:
В данном примере используется метрика силуэта для оценки качества кластеризации. Перебираются различные значения k, после чего выбирается наилучшее.

Итог

Метод k-средних представляет собой базовый, но при этом эффективный инструмент кластеризации, широко применяемый на практике. Его основное преимущество заключается в простоте и скорости работы. Однако необходимость задания числа кластеров и ограничения по форме кластеров требуют аккуратного применения и дополнительного анализа данных.